Northwind 2.0 Developer Edition: Orders

Ala Arman

▶ *Applies To*

Select any topic below to learn about managing Orders in the Northwind Developer Edition.

Overview

This Developer Edition of the Northwind Orders sample application is more advanced than that of the Starter Edition. It expands on the database schema (the tables that are used) and now provides additional advanced features. The intention herein is to introduce you to the features of Microsoft Access, not to run any specific business.

Using Orders

- The Order List is available from the Ribbon. It has a few filter options, and hyperlinks to open each order.
- Both the Order List and the Ribbon have an Add Order button to open a new blank order.
- On a New Order form, select an existing customer from the dropdown. At that point your **Employee** name and the **new** status are selected. The Order Date is already filled in as well. Tax Rate is read from the **SystemSettings** table, and Tax Status defaults from the Customer record.
- New orders and purchase orders are added to the MRU (Most Recently Used) list in the Ribbon. Learn more via the **MRU List** section in this article.
- Leave the **Shipped Date** and **Paid Date** blank for now.
- To add orders for new customers, enter the company name and tab out. The Company Detail form will open to finish the new customer record. Then close it and continue with the order. The new company own.

Was this information helpful? Yes No

- To add items to an order, select a **Product Category** and **Product** for this order and enter Quantity. Unit Price is filled in, and Price is calculated by an expression.
- Advance Order Status and move the order through the workflow from **New > Invoiced > Shipped > Closed** using the buttons at the top of the Order form.
- Invoicing can only happen if product is allocated for that order. If a line item is in No Stock or On Order status, a validation error will occur. The user can create a Purchase Order for that product and receive it, and the order item status will be adjusted to **Allocated**.
- To ship an order the **Shipper** and **Shipping Fee** must be entered. If you forget to do so, a validation error will occur. The **Shipping Fee** is added to the **Order Total.**
- Unshipped orders can be deleted using the **Delete Order** button.
- Order line items cannot be modified after the order is past the **New** status.
- In the Northwind Starter version, the Order process is incredibly simple (e.g., inventory is always available, never runs out, and never has to be purchased). Now, in this Dev edition, a more realistic process addresses at least some such issues. Remember that we are showcasing Access features and best practices, not implementing a real-world application.
- Evidence that we are not implementing a real-world application here includes the fact that dates are not being validated. Hence, it is possible to enter illogical dates such as a Shipped Date that is *before* the Order Date.

Programming Orders

This section addresses noteworthy implementation details of the Order form, **frmOrderDetails**:

The order form gets its data from a simple query **qryOrder** (see **RecordSource** property). Basing a data entry form on a simple one-table query is a best practice. Note that it is not necessary to include **OrderDetails** table in this query. Order Details are handled by the subform.

The OrderList form can open multiple instances of the Order form. This is handy because sales reps deal with lots of interruptions and may need to open another order while working on the first one, or compare it to a third order. The technique is documented here.

The various ID fields get their values from two-column comboboxes: a hidden ID column, and a visible Description column. These comboboxes are bound to simple two-column queries: see the **RowSource** property.

The workflow buttons have business logic associated with them forcing the user to advance the order from 1 to 4. The Northwind Development team is aware that some companies may

Was this information helpful?

use different rules. This would then result in different implementation for the button click events, as well as re-consideration of when an order is definite, and when an order can still be deleted.

The **subform sfrmOrderDetails** is bound to a more complex query. The reasons for that are discussed in the Cascading Comboboxes section below. We check for inventory in the Form_AfterUpdate event when the row is saved and we can run more powerful database queries.

ProductCategory and Product are Cascading comboboxes: selecting from the first (**ProductCategory**) narrows the next one to matching child Product records. The technique used here is described in detail below.

When saving a record, the required fields must be filled out. In Starter edition we let the Access default behavior happen; in this Dev edition a more user-friendly technique is implemented. The technique used here is described in detail below.

For each order line item, the available inventory is checked and the status is set accordingly. The basic idea of this feature is described here.

CASCADING COMBOBOXES

Implementing the Product Category and Product dropdowns as Cascading Comboboxes is tricky because Access does not support this feature out of the box. Four steps are necessary in this technique:

The form must be in Continuous Forms mode (not Datasheet). Textboxes overlap the text portion of each combobox, leaving only their dropdown arrows visible.

The form's record source query, **qryOrderLineItems**, uses the **OrderDetails** table per usual, but also joins with Products and ProductCategories tables to pick up ProductName and **ProductCategoryName**. The two overlapping textboxes are bound to these fields.

The **RowSource** for the Products combobox looks back at **cboProductCategories** to return only products for the category selected in that combobox. Note the syntax "[Form]![cboProductCategories]" in the criteria expression, which is more flexible than the explicit **Forms!FormName!ControlName** syntax, which references one form by name.

After selecting a product category in the unbound **ProductCategories** combobox, its **AfterUpdate** event sets the Products combobox to the first value in its list. This creates a new row in the form's **RecordSource**, which populates the **CategoryName** so it can be displayed by its overlapping textbox.

VALIDATION

Using the validation code implemented in Northwind Dev edition only takes 3 lines of code:

Was this information helpful?

- In Form_BeforeUpdate:
  Cancel = ValidateForm(Me)
- In Form_AfterUpdate and Form_Current:
  ValidateForm_RemoveHighlights Me

Making code very self-contained is a good pattern to follow because it makes it easy to implement everywhere. Professional developers may take this even further, for example, by using form subclassing. (This is beyond the goals for Northwind Dev.)

The form object is passed to the self-contained validation code to validate. It then checks the underlying **RecordsetClone** Fields collection to find out which controls are bound to required fields, and checks if they have a value. If they don't, they are highlighted.

Learn more

- Northwind 2.0 Developer Edition: Things you should know
- Northwind 2.0 Developer Edition: All topics

Need more help?

How can we help you?

Want more options?

Discover **Community**

Explore subscription benefits, browse training courses, learn how to secure your device, and more.

Was this information helpful?



Microsoft 365 subscription benefits



Microsoft 365 training



Microsoft security



Accessibility center

What's new

Surface Pro

Surface Laptop

Surface Laptop Studio 2

Copilot for organizations

Copilot for personal use

AI in Windows

Explore Microsoft products

Windows 11 apps

Microsoft Store

Account profile

Download Center

Microsoft Store support

Returns

Order tracking

Certified Refurbished

Microsoft Store Promise

Flexible Payments

Education

Microsoft in education

Devices for education

Microsoft Teams for Education

Microsoft 365 Education

How to buy for your school

Educator training and development

Deals for students and parents

AI for education

Business

Microsoft AI

Microsoft Security

Dynamics 365

Microsoft 365

Microsoft Power Platform

Microsoft Teams

Microsoft 365 Copilot

Small Business

Developer & IT

Azure

Microsoft Developer

Microsoft Learn

Support for AI marketplace apps

Microsoft Tech Community

Microsoft Marketplace

Marketplace Rewards

Visual Studio

Company

Careers

About Microsoft

Company news

Privacy at Microsoft

Investors

Diversity and inclusion

Accessibility

Sustainability

English (United States) Your Privacy Choices Consumer Health Privacy

Sitemap Contact Microsoft Privacy Terms of use Trademarks Safety & eco Recycling About our ads

© Microsoft 2026

Was this information helpful?